

Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

la

Lucrarea de laborator Nr. 2

Disciplina: Tehnici de programare

Tema: ” Lucrul în linia de comandă Linux ”

A efectuat :

st. gr.SI – 266 Sevostian Daniil

A verificat :

asistent universitar Cristian Goncari

Chișinău – 2026

SARCINA LUCRĂRII

Parcurge pe platforma pwn.college dojo-ul Linux Luminarium și rezolvă integral modulele obligatorii: Hello Hackers, Pondering Paths, Comprehending Commands, Digesting Documentation, File Globbing și Practicing Piping. Rezolvă apoi cel puțin patru dintre modulele recomandate: Shell Variables, Data Manipulation, Processes and Jobs, Untangling Users și Perceiving Permissions. Pe parcursul lucrului notează-ți comenzile exacte pe care le-ai rulat, mesajele de eroare primite și modul în care ai depășit fiecare blocaj, fiindcă întrebările din acest raport se referă la experiența ta directă, nu la definiții. Răspunde apoi, în acest document, la toate cele 20 de întrebări, cu propriile cuvinte și cu exemple concrete din sesiunile tale de lucru. Răspunsurile copiate din manuale sau de pe internet, care nu au legătură cu ceea ce ai rulat efectiv, nu se punctează.

Platforma	Cursul de parcurs	Adresa
pwn.college	Linux Luminarium	https://pwn.college/linux-luminarium/

Modulele pe care le parcurgi

Nr.	Modul	Challenge-uri	De rezolvat	Nivel
1	Hello Hackers	3	3	OBLIGATORIU
2	Pondering Paths	8	8	OBLIGATORIU
3	Comprehending Commands	15	15	OBLIGATORIU
4	Digesting Documentation	7	7	OBLIGATORIU
5	File Globbing	10	10	OBLIGATORIU
6	Practicing Piping	15	15	OBLIGATORIU
7	Shell Variables	8	8	RECOMANDAT
8	Data Manipulation	6	6	RECOMANDAT
9	Processes and Jobs	10	10	RECOMANDAT
10	Untangling Users	4	4	RECOMANDAT
11	Perceiving Permissions	8	8	RECOMANDAT
12	Chaining Commands	12	12	BONUS
13	Terminal Multiplexing	6	6	BONUS
14	Pondering PATH	5	5	BONUS
15	Silly Shenanigans	6	6	BONUS
16	Daring Destruction	5	5	BONUS

Modulele marcate OBLIGATORIU conțin materia pe care se bazează întrebările. Cele marcate RECOMANDAT și BONUS te ajută la întrebările mai dificile.

ÎNTREBĂRI

Răspunde la toate cele 20 întrebări, în chenarele de mai jos. Fiecare răspuns corect valorează 0,5 puncte.

1. La exercițiul Intro to Arguments ai rulat comanda hello mai întâi fără argument, apoi cu argumentul hackers. Explică ce face shell-ul concret cu textul pe care îl scrii la prompt înainte de a porni programul și de ce programul a reacționat diferit în cele două cazuri.

(modulul Hello Hackers)

În exercițiul Intro to Arguments la efectuarea promptului shell-ul a împărțit textul în spații, primul cuvânt fiind comanda, iar celelalte cuvinte după acesta devin argumentele lui. În prima data comanda (*hello*) a fost executat fără argument, astfel programul a dat eroare cerând un argument pentru această comandă. În al doilea caz la execuția programului cu argumentul (*hackers*) aceasta a fost executată corect afișând răspunsul flagul cerut în condiția exercitiului (`pwn.college{03pGiv9uPTqOtylk2EIImFqQFtaS.QX4YjM1wiNxcTMxIzW}`)

2. La exercițiul Command History flag-ul a fost introdus în istoricul shell-ului tău. Relatează concret cum ai ajuns la el: ce ai apăsăat sau ce comandă ai folosit, ce a apărut pe ecran și unde se păstrează acest istoric.

(modulul Hello Hackers)

Pentru a găsi flagul am apăsăat butonul (*Up Arrow*) pentru ca pe ecran să apară comenzile executate mai înainte și a apărut “the flag is `pwn.college{g_Y157qv3Ep6HmY-ngfQBVKYNKF.0lNzEzNxwiNxcTMxIzW}`”, de asemenea în cazul dacă executăm comanda (*history*) în terminal deodată o să se afișeze lista cu toate comenzile executate anterior. Datorită comenzii (*history*) se asigură accesul mai rapid la tot ce se execută pe parcursul utilizării într-o sesiune în `pwn.college`. Istoricul comenzilor se păstrează într-un fișier ascuns în directorul “home” a utilizatorului

3. Aflându-te chiar în directorul /challenge, comanda scrisă doar cu numele programului eșuează, iar varianta cu ./ în față funcționează. Explică ce caută shell-ul în fiecare dintre cele două cazuri.

(modulul Pondering Paths)

La executarea comandai (*pwn*) fără “/” eșuează din cauza că shell-ul caută programul într-o listă fixată unde acest director nu se află. La executarea programului cu (*/pwn*) calculatorul caută directorul pe cale exactă numită “cale absolută” unde se află acesta

4. La exercițiile implicit relative paths, from / și implicit relative path a trebuit să rulezi același program o dată din directorul / și o dată din /challenge, scriind de fiecare dată altceva. Explică de la ce

punct pornește sistemul când primește o cale relativă și cum se schimbă calea pe care trebuie să o scrii atunci când te muți cu cd.

(modulul *Pondering Paths*)

Punctul de plecare pentru o cale relativă este de fiecare dată directorul curent de lucru (*cwd*). În timpul rulării *cd*, locația inițială se schimbă, determinând modul în care este scrisă calea către executabil. Aflat în rădăcină (*/*), comanda (*challenge/run*) pornește din (*/*) și ajunge direct la fișier. Prin mutarea prin (*cd*) în directorul (*/challenge*), nimerim deja în acest director, iar calea scade la fișierul local. Din cauza că Linux ignoră directorul curent din cauza securității, folosim (*./*) pentru a executa (*./run*)

5. La exercițiul home sweet home aveai voie să dai un argument de cel mult trei caractere. Scrie exact ce ai introdus ca argument și explică prin ce mecanism acele câteva caractere ajung să indice un fișier din directorul tău personal.

(modulul *Pondering Paths*)

Argumentul introdus ca cale este (*~/f*). Înainte de executarea comenzii, shell-ul Bash realizează *tilde expansion*, înlocuind simbolul (*~*) cu calea absolută către directorul personal al utilizatorului, adică (*/home/hacker*). Astfel, argumentul scris are doar 3 caractere (*~, /, f*), bash îl extinde automat la calea absolută (*/home/hacker/f*). Programul */challenge/run* primește această cale completă, verifică dacă se află în interiorul directorului personal și dacă argumentul inițial a avut cel mult 3 caractere, după care scrie fișierul cu flagul exact la acea locație.

6. Numele comenzii cat vine de la concatenare. Pe baza exercițiilor pe care le-ai rulat în Comprehending Commands, explică ce se întâmplă când îi dai mai multe fișiere ca argumente și cum se vede acest lucru în ieșirea pe care ai obținut-o.

(modulul *Comprehending Commands*)

Când comandai cat sunt date mai multe fișiere ca argumente, aceasta le citește pe rând, în ordinea exactă în care au fost specificate, și le unește conținutul într-un singur flux continuu afișat în terminal. În ieșirea obținută, acest lucru se vede prin faptul că textul din al doilea fișier apare imediat sub textul din primul fișier, fără spații suplimentare sau delimitatoare între ele. În exemplul din exercițiu “cat: not the pet, but the command!”, la executarea *cat myfile yourfile*, se afișează mai întâi linia din myfile („This is my file!”), urmată direct de linia din yourfile („This is your file!”).

7. La exercițiul An Epic Filesystem Quest ai urmărit indicii dintr-un director în altul până la flag. Relatează concret traseul tău: ce pereche de comenzi ai repetat la fiecare pas și cum ai decis încotro mergi mai departe.

(modulul *Comprehending Commands*)

Traseul a fost parcurs prin aplicarea repetată a perechii de comenzi `cd` (pentru navigarea în directorul indicat) și `ls` (pentru identificarea fișierului ce conținea indiciul), urmată de comanda `cat` pentru citirea acestuia. Decizia privind direcția de urmat s-a bazat direct pe mesajul extras din fiecare fișier, care specifica calea exactă: The next clue is in: /cale.... La fiecare pas au fost respectate condițiile speciale impuse de indicii: pentru indiciile marcate ca *trapped*, citirea s-a făcut direct prin cale absolută fără utilizarea comenzii `cd` pentru a evita autodestrușirea, iar pentru cele marcate ca *hidden*, s-a folosit `ls -a` pentru a localiza fișierele ascunse care începeau cu punct.

Parcursul succesiv al directorilor este :

```
/usr/share/doc/john-data, /usr/share/doc/libmpc3, /usr/include/x86_64-linux-gnu/c++,  
/usr/lib/python3/dist-packages/pip/_internal/resolution/_pycache_, /usr/lib/x86_64-linux-  
gnu/perl-base/unicore/lib/BidiC, /usr/lib/python3/dist-packages/pwnlib/data/includes/android  
și /usr/lib/x86_64-linux-gnu/perl/5.38.2/auto/Encode/KR a dus la citirea fișierului INSIGHT,  
obținându-se astfel flagul final  
"pwn.college{sAeJqHD58NCqmxrLqnKrMh8QBw4.QX5IDO0wiNxcTMxIzW}"
```

8. La exercițiul linking files ai creat o legătură simbolică cu `ln -s`. Explică ce conține de fapt fișierul-legătură și ce se întâmplă în momentul în care rulezi `cat` pe el.

(modulul *Comprehending Commands*)

Legătura simbolică (*symlink*) conține doar o cale de text către fișierul țintă, ocupând o dimensiune infimă pe disc egală cu numărul de caractere al acelei căi. În momentul în care se rulează comanda `cat` pe fișierul-legătură, sistemul de operare detectează tipul special al fișierului, citește calea stocată în interiorul său și redirecționează automat operația de citire către fișierul țintă, afișând conținutul acestuia în terminal.

9. Compară cele trei surse de documentație folosite în *Digesting Documentation*: `man`, opțiunea `--help` și comanda `help`. Spune pentru ce fel de comandă se potrivește fiecare și de ce pentru comanda-builtin din ultimul exercițiu nu ai putut folosi `man`.

(modulul *Digesting Documentation*)

Cele trei surse de documentație se diferențiază prin tipul de comenzi cărora li se aplică și prin modul de funcționare: comanda `man` este destinată utilitatelor și programelor externe din sistem (cum ar fi `ls` sau `cat`), încărcând fișiere de ajutor dedicate; opțiunea `--help` este utilizată pentru majoritatea programelor executabile pentru a afișa rapid o listă scurtă de

argumente direct în terminal; iar comanda `help` este concepută exclusiv pentru comenzile interne ale interpretorului de comenzi (shell builtins), cum ar fi `cd`. Pentru comanda builtin din ultimul exercițiu nu s-a putut folosi `man` deoarece comenzile interne sunt integrate direct în pachetul Bash și nu există ca fișiere executabile separate pe disc, motiv pentru care sistemul nu are pagini de manual individuale asociate pentru ele în baza de date `man`.

10. La exercițiul Searching For Manuals pagina de manual avea un nume generat aleator. Spune ce comandă ai folosit ca să o găsești, după ce cuvânt ai căutat și cum arăta rezultatul afișat de acea căutare.

(modulul Digesting Documentation)

[scrie răspunsul tău aici]

Pentru identificarea paginii de manual cu nume aleatoriu a fost aplicată comanda `man -k` urmată de termenul `challenge`, instrucțiune ce interoghează baza de date a sistemului pentru a returna toate intrările relevante. Căutarea s-a efectuat după cuvântul-cheie **challenge**, identificându-se astfel asocierea cu descrierea de afișare a flagului. Rezultatul returnat de terminal a fost următoarea linie de text:

`snatzoyiz (1) - print the flag!`

11. Explică cine face expandarea unui tipar precum `/challenge/f*`: shell-ul sau programul apelat. Argumentează pe baza a ceea ce ai observat efectiv, inclusiv ce s-a întâmplat când niciun fișier nu se potrivea tiparului.

(modulul File Globbing)

Expandarea unui tipar precum `/challenge/f*` este realizată exclusiv de către **shell** (interfața în linie de comandă, de exemplu `bash`), și nu de către programul apelat.

Atunci când este introdusă o comandă care conține caractere de tip wildcard, shell-ul analizează mai întâi sistemul de fișiere, identifică toate căile care se potrivesc cu tiparul specificat și înlocuiește argumentul cu lista completă a acestora înainte de a executa programul. Prin urmare, programul apelat primește direct căile deja extinse în lista sa de argumente (`argv`), fără a avea cunoștință despre faptul că utilizatorul a folosit un tipar cu caracter special.

În situația în care niciun fișier din sistem nu se potrivește tiparului furnizat, shell-ul transmite argumentul exact așa cum a fost introdus, sub formă de text literal. Programul apelat primește șirul netransformat și încearcă să acceseze un fișier cu denumirea respectivă, returnând o eroare de tipul `No such file or directory`. Faptul că programul primește textul brut doar atunci când nu există potriviri demonstrează că procesul de expandare este gestionat complet de shell înainte de lansarea în execuție a programului.

12. Compară folosirea unui glob cu folosirea completării prin tasta Tab pentru a ajunge la același fișier. Spune în ce situație concretă din modul ai preferat una și în ce situație cealaltă, și de ce completarea prin Tab este considerată mai sigură.

(modulul File Globbing)

Pentru compararea utilizării unui glob cu completarea prin tasta **Tab**, răspunsul este următorul:

Pentru accesarea fișierelor, un **globbing** folosește caractere speciale (*, ?, []) pentru a potrivi unul sau mai multe fișiere deodată, trimițând lista direct către program fără confirmare prealabilă. În schimb, **completarea prin Tab** este un proces interactiv care completează numele exact al unui singur fișier pe ecran în timp real.

În cadrul modulului, **globbing-ul** a fost preferat în nivelurile unde trebuiau transmise mai multe fișiere simultan ca argumente (de exemplu, la *Mixing globs* folosind [cep]*).

Completarea prin Tab a fost preferată la nivelurile cu nume lungi și complexe de fișiere (cum ar fi /challenge/files/pwncollege-flag), unde tastarea manuală era imposibilă sau restricționată.

Completarea prin tasta **Tab** este considerată mai sigură deoarece oferă confirmare vizuală a căii exacte înainte de apăsarea tastei **Enter**, prevenind executarea accidentală a comenzilor pe fișiere nedorite (cum ar fi ștergerea eronată prin rm).

13. Descrie ce se întâmplă concret cu textul dintre două programe legate prin pipe, pe exemplul pe care l-ai rulat tu la exercițiul Grepping live output. Precizează de unde citește al doilea program și dacă textul mai trece prin vreun fișier de pe disc.

(modulul Practicing Piping)

Pentru comanda /challenge/run | grep pwn.college executată în exercițiul **Grepping live output**, descrierea detaliată este următoarea:

La executarea comenzii, prima programă (/challenge/run) își trimite textul generat în ieșirea standard (stdout). Operatorul pipe (|) conectează direct ieșirea standard a primei programe cu intrarea standard (stdin) a celei de-a doua programe (grep). Textul este transmis între cele două procese în timp real („din mers”).

Al doilea program (grep) citește textul direct din intrarea sa standard (**stdin**).

Textul **nu mai trece prin niciun fișier de pe disc**. Transferul de date prin pipe (|) se realizează exclusiv în memoria RAM printr-un buffer gestionat direct de nucleul sistemului de operare

14. Ai redirecționat ieșirea programului /challenge/run într-un fișier folosind operatorul > și totuși pe ecran au continuat să apară mesaje. Explică de ce au apărut acele mesaje și ce ai scris ca să le capturezi și pe ele.

(modulul Practicing Piping)

Mesajele au apărut pe ecran deoarece au fost trimise prin fluxul de erori standard (**stderr** / File Descriptor 2). Operatorul > redirecționează doar ieșirea standard (**stdout** / File Descriptor 1), astfel încât erorile rămân neafectate și sunt afișate direct în terminal. Pentru a le captura și pe ele, am folosit unificarea erorilor cu ieșirea standard: /challenge/run 2>& Prin utilizarea operatorului 2>&1, am redirecționat fluxul de erori (stderr / File Descriptor 2) în ieșirea standard (stdout / File Descriptor 1). În acest mod, ambele fluxuri au fost unificate într-un singur canal și trimise mai departe prin pipe (|) către grep, permițând capturarea și filtrarea tuturor mesajelor.1 | grep pwn.college

15. Modulul îți cere mai întâi să salvezi ieșirea într-un fișier și apoi să o filtrezi cu grep, iar la exercițiul următor să filtrezi direct prin pipe. Compară cele două metode și spune în ce situație reală ai alege salvarea în fișier și în ce situație pipe-ul direct.

(modulul Practicing Piping)

[scrie răspunsul tău aici]

Putem alege filtrarea directă prin pipe atunci când avem nevoie de un rezultat rapid în timp real, fără a ocupa spațiu pe disc, deoarece datele sunt transmise direct din memoria unui proces în altul. Sau putem alege salvarea ieșirii într-un fișier atunci când avem nevoie de o copie permanentă pentru auditare, arhivare sau când analizăm fișiere mari de loguri și vrem să aplicăm mai multe filtre succesive pe aceleași date fără a reexecuta comanda inițială.

16. La exercițiul Exporting Variables ai setat o variabilă, iar programul lansat de tine nu o vedea până când nu ai folosit export. Explică diferența dintre o variabilă de shell obișnuită și una exportată, din perspectiva procesului-copil.

(modulul Shell Variables)

Atunci când un shell lansează în execuție un program, acesta creează un proces-copil, iar din perspectiva acestuia există o diferență fundamentală între o variabilă de shell obișnuită și una exportată. O variabilă de shell obișnuită (locală) există exclusiv în spațiul de memorie al procesului-părinte (shell-ul curent). În momentul în care lansezi o nouă comandă sau un program, sistemul de operare creează un proces-copil izolat, care nu moștenește variabilele locale ale părintelui său. Din acest motiv, programul /challenge/run nu a putut accesa inițial variabila, fiind complet invizibilă pentru el. În schimb, o variabilă exportată prin comanda export este marcată pentru a fi inclusă în blocul mediului de executare (*environment block*). La crearea procesului-copil (prin apelurile de sistem fork și exec), sistemul de operare copiază această tabelă cu variabile de mediu în noul proces. Astfel, din perspectiva

procesului-copil, o variabilă exportată este moștenită automat și devine accesibilă în propriul său mediu de lucru.

17. La exercițiile din Data Manipulation flag-ul îți venea stricat, cu caractere în plus, cu linii noi intercalate sau cu majusculile inversate. Alege unul dintre aceste exerciții, scrie comanda exactă pe care ai folosit-o pentru a repara ieșirea și spune cum arăta textul înainte și după.

(modulul Data Manipulation)

Am ales exercițiul Translating Characters.

Am folosit comanda (/challenge/run) pentru a primi flagul cu majusculile inversate (PWN.COLLEGE{4M5bW-ECbz66YtI7k8TcmND0BP2.01mXeZnXWInXCtmXiZw}).

Dupa aceasta am utilizat comanda (/challenge/run | tr 'a-zA-Z' 'A-Za-z') și am primit flagul correct (pwn.college{4m5Bw-ecBZ66yTi7K8tCMnd0bp2.01MxEzNxwiNxcTMxIzW})

18. Ai apăsă Ctrl-Z peste un program care rula. Explică în ce stare ajunge procesul imediat după această apăsare și de ce el nu își continuă lucrul până când nu dai comanda bg sau fg.

(modulul Processes and Jobs)

Atunci când apeșăm combinația de taste **Ctrl-Z**, terminalul trimite procesului din prim-plan semnalul **SIGTSTP** (*Terminal Stop*). În acel moment, procesul trece instantaneu în starea **Stopped**. Procesul nu își mai continuă activitatea deoarece nucleul Linux îi suspendă complet timpul de procesare CPU. Deși rămâne încărcat în memoria RAM, execuția instrucțiunilor sale este înghețată, iar shell-ul îți redă controlul asupra terminalului. Pentru a relua execuția, procesul are nevoie de semnalul **SIGCONT**. Comanda **fg** aduce procesul înapoi în prim-plan, iar comanda **bg** îi permite să continue rularea activă în fundal, lăsând terminalul liber pentru alte comenzi.

19. La exercițiul The SUID Bit ai adăugat bitul SUID unui program și abia apoi el a putut citi flag-ul. Explică cu ce identitate rulează un program obișnuit față de unul cu bit SUID și de ce asta schimbă rezultatul verificării drepturilor la citirea fișierului.

(modulul Perceiving Permissions)

Modul în care un proces interacționează cu fișierele protejate depinde de identitatea sub care rulează în sistem:

Un program obișnuit se execută cu identitatea și permisiunile utilizatorului care îl lansează (*Real User ID*). Dacă este pornit de utilizatorul hacker, procesul va avea doar drepturile

acestui. În schimb, un program cu bitul SUID activat (chmod u+s) se execută temporar cu identitatea **proprietarului fișierului executabil** (*Effective User ID*), indiferent de cine îl pornește.

Această diferență schimbă direct rezultatul la citirea fișierului /flag. Deoarece /flag aparține utilizatorului root, un program obișnuit rulat de hacker primește eroarea *Permission denied*. După adăugarea bitului SUID pe /challenge/getroot, sistemul de operare vede că solicitarea vine de la un proces care rulează cu drepturile proprietarului său (root), permițând astfel citirea fișierului protejat.

20. În modulul Untangling Users ai schimbat identitatea o dată cu su și o dată cu sudo. Explică ce parolă ți s-a cerut în fiecare dintre cele două cazuri și pe ce se bazează sistemul când decide dacă ți permite sau nu operația.

(modulul Untangling Users)

Atunci când schimbăm identitatea de lucru într-un sistem Linux, comenzile su și sudo folosesc concepte diferite de autentificare și autorizare:

Comanda su (*substitute user*) îți cere **parola contului la care încerci să comuți** (de exemplu, parola utilizatorului root sau a utilizatorului zardus). Sistemul decide dacă ne permite accesul printr-o simplă verificare a cunoașterii acelui secret: compară hash-ul parolei introduse cu cel stocat securizat în fișierul /etc/shadow. Dacă acestea potrivesc, su (care este un executabil cu bitul SUID setat) îți deschide un shell în numele noului utilizator.

În schimb, comanda sudo (*substitute user, do*) cere **parola propriului cont al utilizatorului**. Decizia sistemului nu depinde de cunoașterea parolei de root, ci de o politică de autorizare definită de administrator în fișierul /etc/sudoers. Sistemul verifică dacă utilizatorul tău are dreptul explicit de a rula acea comandă cu privilegii elevate, oferindu-ți acces pe baza permisiunilor atribuite contului tău, nu pe baza distribuirii parolei de administrare.

CONCLUZII

Parcurgerea acestor exerciții din modulele Linux Luminarium mi-a oferit o înțelegere practică profundă a sistemului de operare. Am învățat să transform și să curăț fluxuri de date folosind utilitarul `tr`, să gestionez stările proceselor prin semnalele `SIGTSTP` (`Ctrl-Z`) și `SIGCONT` (`bg/fg`), precum și să controlez privilegiile prin intermediul bitului `SUID`, al comutării cu `su` și al politicilor de autorizare din `sudo`.

Cel mai dificil moment l-am întâmpinat la sarcina de spargere a parolelor cu John the Ripper, unde a trebuit să procesez corect structura fișierului `/etc/shadow` și să interpretez hash-urile generate. De asemenea, m-am blocat serios la conceptele de **file globbing**, în special atunci când a trebuit să potrivesc modele complexe de fișiere folosind caractere speciale precum `*`, `?` sau parantezele drepte `[]`. Blocajul a apărut din cauza extinderilor neașteptate făcute de shell înainte ca comenzile să primească argumentele, ceea ce ducea la erori de sintaxă sau la selectarea incorectă a fișierelor.

Am ieșit din acest blocaj testând incremental fiecare mască de cautare cu ajutorul comenzii `echo` pentru a vedea exact ce fișiere erau expandate de shell înainte de a rula comanda finală. Această abordare metodică mi-a dezvoltat atenția la detalii și o înțelegere mult mai clară a modului în care CLI-ul procesează instrucțiunile.

BIBLIOGRAFIE

1. pwn.college (Arizona State University), Linux Luminarium, [Online]. Disponibil: <https://pwn.college/linux-luminarium/>
2. pwn.college, linux-luminarium (codul sursă și descrierile modulelor și ale challenge-urilor), GitHub, [Online]. Disponibil: <https://github.com/pwncollege/linux-luminarium>
3. Free Software Foundation, GNU Bash Reference Manual, [Online]. Disponibil: <https://www.gnu.org/software/bash/manual/bash.html>
4. M. Kerrisk, The Linux man-pages project (paginile de manual ale comenzilor Linux), [Online]. Disponibil: <https://man7.org/linux/man-pages/>
5. W. Shotts, The Linux Command Line: A Complete Introduction, No Starch Press, [Online]. Disponibil: <https://linuxcommand.org/tlcl.php>
6. The Linux Foundation, Filesystem Hierarchy Standard (FHS), versiunea 3.0, [Online]. Disponibil: https://refspecs.linuxfoundation.org/FHS_3.0/fhs/index.html